Université Paris 8 –
Vincennes-Saint-Denis

Société Générale

**Master 2 — Cybersécurité et Sciences des Données**

Apprenticeship Report

Academic Year 2025–2026

# Automating Identity Governance and Operational Security in a DevOps Banking Environment

**BENSAADA Manar**

**Academic Supervisor**

M.BOUBCHIR Larbi
Université Paris 8

**Company Supervisor**

M. CHEBEL Ali
Société Générale — DevOps/SRE Team

**Host Organization:** Société Générale, La Défense, France
**Team:** DevOps / Site Reliability Engineering (SRE)
**Defense date:** June 23, 2026

# Acknowledgements

I would like to express my sincere gratitude to the people who supported me throughout this apprenticeship and the writing of this report.

First, I wish to thank **Ali CHEBEL**, my company supervisor at Société Générale, for his guidance, availability, and the trust he placed in me from the very beginning of my internship through the full duration of the work-study period. His technical expertise and constructive feedback were invaluable in shaping both my work and my professional development.

I also thank the entire **DevOps/SRE team** at Société Générale — in Paris, Montréal, and Bangalore — for their warm welcome, their collaborative spirit, and for sharing their knowledge on a daily basis. Working in such an international and technically demanding environment has been a uniquely enriching experience.

I am grateful to **Larbi BOUBCHIR**, my academic supervisor at Université Paris 8, for his pedagogical support and for the framework provided by the Master 2 Cybersécurité et Sciences des Données programme, which gave me the theoretical foundations necessary to approach complex technical challenges with rigor and critical thinking.

Finally, I extend my thanks to the members of the jury for the time they dedicate to reading and evaluating this report.

# Abstract

Large banking organizations operate complex, multi-tool DevOps platforms where the management of user identities and access rights represents both a critical security challenge and a significant operational burden. This report focuses on the Master 2 apprenticeship, which builds upon a previous Master 1 internship conducted in the same DevOps/SRE team of Société Générale, a leading European banking group.

The central problem addressed is the following: *how can the automation of identity governance and operational security processes — through CI/CD pipelines and workflow orchestration — reduce human error, improve regulatory compliance, and increase platform resilience in a critical banking environment?*

Four main contributions are presented. First, the industrialization and migration of the DUAR (Detailed User Access Review) process from Jenkins to Apache Airflow, enabling daily automated access reviews across ten DevOps applications. Second, the design and production deployment of a GitHub Actions pipeline automating the revocation of leaver accounts across multiple platforms, reducing processing time from several hours to under two minutes per batch. Third, the development of a suite of Airflow DAGs covering operational workflows including file storage backups, change and incident management, and a CI/CD pipeline for automated DAG deployment. Fourth, the modernization of legacy server infrastructure through a Windows Server Gen1 to Gen2 virtual machine migration.

All contributions are implemented in a regulated banking context, with full integration into the team's Agile (SAFe) workflow and ITSM change management process. The results demonstrate measurable improvements in security posture, operational efficiency, and audit traceability.

**Keywords:** DevOps, Site Reliability Engineering, IAM, Identity Governance, Apache Airflow, GitHub Actions, CI/CD, Leaver Management, DUAR, Access Review, Automation, DevSecOps, Banking Security.

# Résumé

Les grandes organisations bancaires opèrent des plateformes DevOps complexes et multi-outils, au sein desquelles la gestion des identités et des droits d'accès constitue à la fois un enjeu de sécurité critique et une charge opérationnelle significative. Ce rapport se concentre sur l'alternance de Master 2, qui s'inscrit dans la continuité d'un stage de Master 1 réalisé au sein de la même équipe DevOps/SRE de Société Générale, l'un des principaux groupes bancaires européens.

La problématique centrale est la suivante : *comment l'automatisation des processus de gouvernance des identités et de sécurité opérationnelle — via des pipelines CI/CD et l'orchestration de workflows — peut-elle réduire les erreurs humaines, améliorer la conformité réglementaire et renforcer la résilience des plateformes dans un environnement bancaire critique ?*

Quatre contributions principales sont présentées. En premier lieu, l'industrialisation et la migration du processus DUAR (*Detailed User Access Review*) de Jenkins vers Apache Airflow, permettant la génération automatique quotidienne des revues d'accès pour une dizaine d'applications. En deuxième lieu, la conception et le déploiement en production d'un pipeline GitHub Actions automatisant la révocation des accès des utilisateurs sortants (*leavers*) sur plusieurs plateformes, réduisant le temps de traitement de plusieurs heures à moins de deux minutes par lot. En troisième lieu, le développement d'un ensemble de DAGs Airflow couvrant des workflows opérationnels — sauvegardes de stockage, gestion des changements et des incidents — ainsi qu'un pipeline CI/CD pour le déploiement automatisé des DAGs. En quatrième lieu, la modernisation d'une infrastructure legacy par la migration d'une machine virtuelle Windows Server de la génération Gen1 vers Gen2.

L'ensemble de ces contributions a été réalisé dans un cadre bancaire réglementé, en intégration complète avec la méthodologie Agile SAFe et les processus de gestion des changements ITSM. Les résultats démontrent des améliorations mesurables en matière de sécurité, d'efficacité opérationnelle et de traçabilité des audits.

**Mots-clés :** DevOps, SRE, IAM, gouvernance des identités, Apache Airflow, GitHub Actions, CI/CD, gestion des leavers, DUAR, revue d'accès, automatisation, DevSecOps, sécurité bancaire.

# Contents

# List of Figures

# List of Tables

# List of Acronyms

**DAG** Directed Acyclic Graph.

**DUAR** Detailed User Access Review.

**ITSM** IT Service Management.

**SAFe** Scaled Agile Framework.

CHAPTER 1

# General Introduction

The financial sector is one of the most demanding environments for information technology. Banking institutions operate under strict regulatory frameworks, serve millions of customers with near-zero tolerance for downtime, and maintain complex, multi-layered software platforms that evolve continuously under the pressure of digital transformation. Within this context, the DevOps and Site Reliability Engineering (SRE) disciplines have emerged as critical enablers, allowing large organizations to deliver reliable software at scale while maintaining operational security and regulatory compliance.

This report documents the work carried out during Master 2 apprenticeship, which builds upon a previous Master 1 internship (*alternance*) within the **DevOps/SRE team of Société Générale**, one of Europe's largest banking groups. Over this period, the primary focus was the design, implementation, and production deployment of automation solutions targeting two interrelated challenges: **identity governance** — ensuring that the right people have the right access to the right tools at all times — and **operational security** — reducing the attack surface and manual toil associated with managing a complex, heterogeneous DevOps platform in a regulated banking environment.

## 1.1 Context and Motivation

Modern DevOps platforms in large banking institutions are rarely monolithic. They are composed of tens of interdependent tools covering the full software delivery lifecycle: source code management, continuous integration and delivery, artifact storage, deployment orchestration, code quality analysis, and infrastructure monitoring. Each of these tools maintains its own user base, its own authentication mechanism, and its own access control model. Managing this heterogeneous ecosystem — and keeping it secure — is a non-trivial operational challenge.

Two recurring failure modes characterize the security posture of such environments. The first is **access sprawl**: over time, users accumulate permissions across tools and instances that exceed their current operational needs, violating the principle

of least privilege [1]. The second is **orphaned accounts**: when an employee or contractor leaves the organization, their accounts on DevOps platforms — holding active credentials and potentially elevated permissions — are not always revoked promptly or exhaustively. Both failure modes represent exploitable vulnerabilities that are well-documented in the security literature and actively targeted by threat actors [2].

In parallel, the growing complexity of operational workflows — backups, reporting, change management, access reviews — creates significant **toil**: repetitive, manual, automatable work that consumes engineering time without adding long-term value [3]. In an SRE team responsible for the reliability and security of a critical banking platform, reducing toil through automation is not merely an efficiency objective; it is a prerequisite for maintaining the quality of service that the banking context demands.

These two challenges — identity governance and operational toil — motivated the technical agenda of this apprenticeship and constitute the backbone of the contributions presented in this report.

## 1.2 Professional Context of the Apprenticeship

This apprenticeship was carried out at **Société Générale**, a French multinational banking and financial services corporation headquartered in Paris, within its **DevOps/Site Reliability Engineering (SRE) team**. The team is responsible for the reliability, security, and continuous improvement of the DevOps platform used by software engineering teams across the organization.

The work-study contract (*contrat d'apprentissage*) spans the academic year 2024–2026, corresponding to the **Master 2 Cybersécurité et Sciences des Données** programme at Université Paris 8. This period is preceded by a five-month internship (*stage*) carried out at the same organization during the Master 1 year (2024), which focused on the initial design and implementation of a leaver account revocation automation pipeline. The M2 apprenticeship extends, industrializes, and significantly broadens the scope of that foundational work, while introducing new contributions in workflow orchestration, CI/CD automation, and infrastructure modernization.

Throughout the apprenticeship, the author occupied the role of **DevOps/SRE Engineer**, participating in all aspects of the team's operational life: daily standups, sprint planning, PI planning sessions, production change management, and collaboration with the IAM, Security, and application teams. All contributions were delivered within the team's Scaled Agile Framework (SAFe) organizational framework, man-

aged via Jira, and subject to the organization's IT Service Management (ITSM) change governance process.

## 1.3 Problem Statement

The central challenge addressed in this report can be formulated as follows:

**Problem Statement**

*In a large banking institution operating a complex, heterogeneous, multi-tool DevOps platform, how can the systematic automation of identity governance and operational security processes — through workflow orchestration and CI/CD pipelines — reduce human error, improve regulatory compliance, and increase platform resilience, while remaining compatible with the organization's change management and audit requirements?*

This problem statement encompasses three distinct but interrelated tensions that any practitioner in a similar context must navigate:

1. **Security vs. operational velocity.** Rigorous access governance requires frequent, comprehensive reviews and prompt revocation of orphaned accounts. But in a large organization, the volume and heterogeneity of platforms make this process prohibitively expensive if performed manually. Automation resolves this tension — but only if the automated solution is itself trustworthy, auditable, and resistant to misconfiguration.
2. **Automation vs. control.** Introducing automated pipelines into a production banking environment raises new risks: a misconfigured automation may delete legitimate accounts, trigger cascading failures, or create audit gaps. The solution must embed controls — dry-run modes, peer review gates, structured logging, and ITSM integration — that make automated actions as accountable as manual ones.
3. **Standardization vs. heterogeneity.** Each tool in the DevOps ecosystem exposes a different API, a different user model, and a different authentication mechanism. A scalable governance solution must abstract over this heterogeneity without losing per-tool precision.

Addressing these tensions in a real production environment, under regulatory constraints and with measurable outcomes, constitutes the core of the work presented in this report.

## 1.4 Objectives and Main Contributions

### Objectives

The apprenticeship pursued four operational objectives, each corresponding to a concrete gap identified in the team's existing processes:

**O1. Industrialize access reviews.** Migrate the Detailed User Access Review (DUAR) process from a fragmented set of Jenkins jobs to a unified, observable, and dynamically extensible Apache Airflow pipeline covering all DevOps platform applications.

**O2. Automate leaver account revocation.** Extend and productionize the GitHub Actions pipeline initiated during the M1 internship to cover all critical DevOps platforms, and establish a regular cadence of production change execution with full ITSM traceability.

**O3. Automate operational workflows.** Replace manual and ad-hoc operational tasks — file storage backups, change and incident reporting, data archiving — with scheduled, monitored Airflow DAGs, and provide the team with a CI/CD pipeline for safe DAG deployment.

**O4. Contribute to infrastructure modernization.** Support the team's infrastructure roadmap by executing a Windows Server virtual machine migration from Generation 1 to Generation 2, following the organization's change governance framework.

### Main Contributions

The following contributions were designed, implemented, and deployed to production during the apprenticeship period:

**Contribution 1 — DUAR Migration and Industrialization**

Design and deployment of a dynamic Apache Airflow DAG replacing a fragmented set of Jenkins jobs for daily user access review generation across ∼10 DevOps applications. Includes SGConnect OAuth2 integration, standardized output formatting, and a scalable connector architecture enabling new application onboarding with zero DAG logic changes.

**Contribution 2 — Leaver Access Revocation Automation**

Production deployment of a modular Python/GitHub Actions pipeline automating the revocation of leaver accounts across SGitHub, JFrog Artifactory, TeamCity, Jenkins, XLDeploy, and ArgoCD, with ongoing extension to SonarQube and

Nexus. Processing time reduced from several hours to under two minutes per batch. All executions are governed by Unity ITSM change requests.

**Contribution 3 — Operational Airflow DAG Suite and CI/CD Pipeline**

Development of eight production Airflow DAGs covering file storage backups (Jira, XLDeploy, Jenkins, GitHub Runner), change and incident management reporting, and data archiving. Design and deployment of a team-wide CI/CD pipeline automating DAG deployment from the SGitHub repository to the Airflow instance, with syntax validation, branch-based environment targeting, and audit-ready execution logs.

**Contribution 4 — Infrastructure Modernization**

Execution of a Windows Server 2016 virtual machine migration from Generation 1 (BIOS) to Generation 2 (UEFI) on the Société Générale private cloud, following the full ITSM change lifecycle including pre-migration documentation, development validation, and production cutover.

## 1.5 Report Structure

The remainder of this report is organized as follows:

**Chapter 2 — Professional and Organizational Context** presents the host organization, the regulatory constraints of the banking sector, the DevOps/SRE team's structure and responsibilities, the technical ecosystem and tooling landscape, and the Agile and ITSM governance frameworks within which all contributions were delivered.

**Chapter 3 — Applied Technical Background** provides the theoretical and conceptual foundations directly relevant to the contributions: identity and access management, DevSecOps and SRE principles, workflow orchestration with Apache Airflow, and CI/CD pipeline design in regulated production environments.

**Chapter 3 — Engineering Approach and Methodology** describes the analytical and engineering approach adopted throughout the apprenticeship: the as-is process analysis method, the design principles governing all contributions, the implementation and validation strategy, and the confidentiality framework applied to this report.

**Chapter 4 — Contributions and Implemented Solutions** constitutes the technical core of the report. It presents, in detail, the four contributions described above, with

architecture descriptions, design decisions, implementation specifics, and measured outcomes for each.

**Chapter 5 — Results, Critical Analysis and Perspectives** synthesizes the measurable impact of the contributions, discusses their limits and the risks introduced by automation, and situates the work within a broader perspective of future improvements.

**Chapter 6 — Conclusion** summarizes the contributions, revisits the problem statement, reflects on the skills acquired, and outlines the professional and technical outlook.

CHAPTER 2

# Professional and Organizational Context

## 2.1 Société Générale and the Banking IT Context

Founded in 1864, **Société Générale** is one of Europe's largest banking groups, employing over 117,000 people across 62 countries. As a systemically important financial institution, it operates under strict regulatory oversight (ECB, ACPR, AMF) with binding requirements on IT continuity, security, and auditability — most recently reinforced by the  (), in force since January 2025 [4].

These constraints translate directly into engineering requirements: every production action must be documented, approved, and traceable; every system holding user data must enforce access controls; and every process touching security must produce audit-ready evidence. This is the environment in which all contributions of this report were designed and deployed.

## 2.2 The DevOps/SRE Team

The apprenticeship was conducted within the **DevOps/SRE** (Site Reliability Engineering) team, responsible for the reliability, security, and continuous improvement of the software delivery platform used by engineering teams across the organization. The team comprises approximately 25 engineers distributed across Paris (primary site), Montréal, and Bangalore, enabling near-24/7 operational coverage. Daily standups via videoconference maintain coordination across time zones.

The team's mission spans four axes: **platform reliability** (CI/CD, artifact management, deployment tools), **security and compliance** (access governance, user lifecycle management), **automation and toil reduction** (replacing manual operations with monitored workflows), and **enablement** (supporting development teams in using the platform). These axes map directly onto the contributions described in Chapter 4.

## 2.3 Technical Ecosystem and Tooling Landscape

The platform covers the full software delivery lifecycle through approximately fifteen tools. Table 2.1 presents the ecosystem organized by function, with explicit linkage to the contributions that directly involve each tool.

| Domain | Tool(s) | Role |
|---|---|---|
| Source control | SGitHub Enterprise | Code, configurations and pipeline definitions. |
| CI/CD | Jenkins, TeamCity | Build automation across multiple instances. |
| | GitHub Actions | Workflow automation and CI/CD pipelines. |
| Artifacts | JFrog Artifactory | Primary artifact repository. |
| | Sonatype Nexus | Secondary artifact repository. |
| Deployment | ArgoCD | GitOps-based Kubernetes delivery. |
| | XLDeploy | Release automation for non-Kubernetes deployments. |
| Code quality | SonarQube | Static analysis, code quality and vulnerability detection. |
| Logging | ELK Stack | Centralized log aggregation. |
| Orchestration | Apache Airflow | DAG-based workflow scheduling and monitoring. |
| Monitoring | Prometheus, Grafana | Metrics collection, dashboards and observability. |
| | Alerta | Incident alert routing. |
| ITSM | Unity | Change requests and incident tracking. |
| Secrets | HashiCorp Vault | Centralized credential management. |
| | GitHub Secrets | Secret injection for CI/CD pipelines. |
| Auth | SGConnect (OAuth2) | Internal API authentication. |
| | LDAP / Active Directory | Identity backbone for platform users and access control. |

**Table 2.1:** DevOps/SRE platform tooling landscape

Two characteristics of this ecosystem are particularly relevant to the contributions in this report. First, **heterogeneity**: each tool exposes a different API, a different user model, and a different authentication mechanism. This makes centralized access governance non-trivial — it is precisely the problem addressed by the DUAR pipeline and the leaver revocation automation. Second, **multi-instance deployment**: tools such as Jenkins and TeamCity are deployed on multiple independent instances serving different business units. A user may hold accounts on several instances simultaneously, which manual processes handle poorly. The automated solutions described in Chapter 4 resolve this through instance-level metadata derived from the DUAR extract.

**Apache Airflow** deserves specific mention as the platform's workflow orchestration layer, given its central role in three of the four contributions. Airflow defines workflows as Directed Acyclic Graph (DAG)s in Python, provides a native scheduler, and exposes a web UI for real-time monitoring of all workflow executions.

## 2.4 Agile Governance and ITSM Change Management

### Agile at Scale: SAFe

The team operates within the Scaled Agile Framework (SAFe) framework [5], structured around **Program Increments (PIs)** of approximately ten weeks, each divided into five two-week sprints. PI planning events align the team on priorities, surface inter-team dependencies, and produce a visible delivery plan. The author participated in two PI planning cycles, contributing to the decomposition of the DUAR migration and leaver revocation epics into user stories tracked in Jira.

### ITSM Change Management

Every production action — whether manual or automated — must be preceded by a formal **change request** on the Unity ITSM platform, documenting scope, implementation plan, risk assessment, and rollback procedure. The change is reviewed and approved before execution; post-execution, it is updated with actual results and formally closed.

This requirement applies without exception to the automated pipelines in this report. The leaver revocation pipeline, for instance, does not execute autonomously: each run is preceded by an approved Unity change request, and the pipeline's execution logs are attached to the change record as post-implementation evidence. Rather than conflicting with automation, this governance process is embedded into it — making automated actions more traceable than equivalent manual operations.

## 2.5 Scope and Position of the Apprenticeship

Within the team, I held the role of DevOps/SRE Engineer Apprentice during the M2 apprenticeship period, following a previous M1 internship carried out in the same environment. The initial internship focused on the first version of the leaver access revocation automation, while the apprenticeship extended this work and broadened my scope to the team's daily DevOps/SRE activities.

Over time, I became fully integrated into the team's operational workflow and progressively took part in regular tasks related to automation, access governance, Airflow workflows, CI/CD pipelines, production changes and infrastructure support. This allowed me to develop a practical understanding of how DevOps/SRE practices are applied in a regulated banking environment.

My contributions were therefore not limited to a single isolated project. They combined the continuation and industrialization of the leaver revocation project with

several operational and security-oriented improvements carried out in collaboration with the Security, IAM, infrastructure and SRE teams. All implemented solutions were reviewed, validated and integrated into the team's existing governance and production processes.

CHAPTER 3

# Technical Background and Methodology

This chapter presents the technical foundations required to understand the contributions described in the next chapter. It does not aim to provide an exhaustive theoretical study of DevOps, SRE or cybersecurity. Instead, it introduces the main concepts used throughout the report: identity and access management, access reviews, leaver revocation, CI/CD pipelines, workflow orchestration, secrets management and production governance.

The second part of the chapter presents the engineering methodology followed during the apprenticeship, from the analysis of existing processes to development, validation, peer review and controlled production deployment.

## 3.1 Technical Foundations

This section introduces the main technical notions used throughout the report. The objective is not to give a theoretical overview of each domain, but to clarify the concepts that are directly connected to the implemented contributions.

### 3.1.1 Identity and Access Management

Identity and Access Management (IAM) covers the processes used to manage users, accounts and permissions inside an information system. In a DevOps/SRE environment, this subject is critical because users may have access to sensitive tools such as source code repositories, CI/CD platforms, artifact repositories, deployment tools and monitoring systems.

A key principle in IAM is the **principle of least privilege**: each user or service account should only have the permissions required for its role. In practice, this is difficult to maintain over time. Users may change teams, projects or responsibilities, while old permissions remain active. This can lead to access accumulation and increase the attack surface.

Two IAM-related processes are central in this report:

- **Access reviews**, through the DUAR process, which provide visibility on who has access to which DevOps platforms.
- **Leaver revocation**, which consists of removing accounts and permissions when users leave the organization or no longer require access.

Both processes become complex in a multi-tool environment because each platform has its own users, roles, APIs and permission model. This complexity explains why automation was necessary.

### 3.1.2 DevOps, SRE and Toil Reduction

DevOps promotes collaboration between development, operations and support teams through automation, monitoring and continuous delivery practices. Site Reliability Engineering (SRE) applies software engineering principles to operations in order to improve reliability, reduce incidents and make operational work more repeatable.

One important SRE concept is **toil**: repetitive, manual and automatable work that consumes time without creating long-term value. During the apprenticeship, several tasks matched this definition, such as manual leaver revocation, fragmented DUAR executions, manual DAG deployment and recurring operational workflows.

The goal of automation was therefore not only to save time. It was also to reduce human error, improve consistency and make executions easier to trace and audit.

### 3.1.3 CI/CD and Workflow Orchestration

CI/CD pipelines automate the validation, execution and deployment of software or operational scripts. In this report, GitHub Actions was used in two main ways: to execute the leaver revocation pipeline and to automate the deployment of Airflow DAGs.

Apache Airflow was used for recurring workflows that require scheduling, task dependencies, logs, retries and execution history. In Airflow, a workflow is defined as a DAG, composed of tasks and dependencies. This makes it more suitable than isolated scripts or fragmented Jenkins jobs for operational processes that must run regularly and remain observable.

### 3.1.4 Secrets Management and Production Governance

Automation often needs credentials to call APIs or interact with production tools. For this reason, secrets management is essential. Credentials must not be stored

| Tool / concept | Main role | Use in this report |
|---|---|---|
| GitHub Actions | Pipeline execution and automation | Leaver revocation and DAG deployment. |
| Apache Airflow | Workflow orchestration and scheduling | DUAR migration, backups and operational workflows. |
| Jenkins | Existing CI/CD and job execution platform | Initial DUAR jobs before migration to Airflow. |

**Table 3.1:** Main automation tools used in the report

directly in source code, configuration files or execution logs. In the implemented solutions, secrets were injected at runtime through secure mechanisms such as GitHub Secrets or Vault.

In a banking environment, automation must also remain compatible with production governance. A pipeline that modifies production data or user access cannot bypass validation, approval or audit requirements. The solutions described in this report were therefore designed to produce execution evidence: logs, workflow runs, artifacts and references to ITSM change records when required.

This combination of automation, secrets management and governance is essential: the objective is not only to automate operations, but to automate them in a secure, controlled and auditable way.

## 3.2 Engineering Principles Applied

The same engineering principles were applied across the different contributions. They helped guide implementation choices and ensure that the solutions were not only functional, but also maintainable, secure and compatible with production constraints.

## 3.3 Implementation and Validation Approach

The implementation followed a progressive approach from development to production. This avoided deploying sensitive automation directly into a production environment without sufficient validation.

1. **Development:** scripts, DAGs and connectors were developed in branches and tested with controlled or anonymized data.

| Principle | Application in the project |
|---|---|
| Modularity | Each platform-specific integration was isolated in a connector or module. This made the solutions easier to test, maintain and extend. |
| Security by design | Credentials were not hard-coded. Secrets were injected at runtime through secure mechanisms such as GitHub Secrets or Vault. |
| Traceability | Each execution produced logs, results and evidence that could be reviewed or attached to an ITSM change record. |
| Fail-safe behavior | Missing or invalid inputs caused explicit failures instead of silent errors. Platform-specific errors were logged for follow-up. |
| Integration with existing workflows | Automation was integrated into the team's existing Git, pull request, Airflow and ITSM practices instead of introducing isolated processes. |

**Table 3.2:** Engineering principles applied across the contributions

2. **Non-production validation:** workflows were executed in development environments to verify authentication, API behavior, output format and error handling.
3. **Peer review:** code changes were submitted through pull requests and reviewed before integration.
4. **Production change approval:** sensitive executions were performed only after validation through the internal ITSM change process.
5. **Post-execution control:** execution logs and results were reviewed and, when required, attached to the corresponding change record.

**Validation objective**

The objective was not only to prove that the automation worked, but also to prove that it could be safely executed, understood by the team, and audited afterwards.

## 3.4 Confidentiality and Anonymization

Because the work was carried out in a banking environment, confidentiality constraints were applied throughout the report. Internal hostnames, IP addresses, API endpoints, user data, leaver lists and DUAR extracts are not disclosed.

Code examples, diagrams and technical descriptions are anonymized or simplified when necessary. This makes it possible to explain the engineering approach and the technical contributions while respecting the confidentiality of the host organization.

CHAPTER 4

# Contributions and Implemented Solutions

This chapter presents the technical contributions carried out during the apprenticeship. They span four operational domains: access governance automation, operational workflow automation, infrastructure CI/CD, and infrastructure modernization. Despite their apparent diversity, all contributions share a common intent: replace manual, fragile, or non-existent processes with automated, observable, and audit-ready solutions that meet the security and reliability standards of a regulated banking environment.

## 4.1 Overview and Common Design Thread

Table 4.1 provides a consolidated view of all contributions, their status at the time of writing, and the primary technology involved.
Across all contributions, four design principles were applied consistently (detailed in Chapter 3): **modularity** (each component is independently testable and replaceable), **security by design** (no hard-coded credentials, least-privilege service accounts, Vault and GitHub Secrets for runtime injection), **traceability** (every automated action produces structured logs attached to an ITSM change record), and **fail-safe behavior** (missing inputs abort execution; per-platform errors are caught and logged without canceling the full pipeline).

| # | Contribution | Status | Core technology |
|---|---|---|---|
| §4.2 | DUAR: access review migration and industrialization | In production | Airflow, Python, S3 |
| §4.3 | Leaver access revocation automation | In production | GitHub Actions, Python |
| §4.4.1 | File storage backup automation | In production | Airflow, Azure File Storage |
| §4.4.2 | Bond Weather operational data pipeline | In production | Airflow, Strapi, Unity ITSM |
| §4.5 | CI/CD pipeline for automated DAG deployment | In production | GitHub Actions, Airflow |
| §4.4.3 | Artifactory artifact archiving DAGs | In development | Airflow, JFrog API, S3 |
| §4.6 | Windows Server Gen1 to Gen2 VM rebuild | In development | SG Private Cloud |

**Table 4.1:** Summary of contributions — status and technology

## 4.2 DUAR: Access Review Industrialization and Migration to Airflow

### 4.2.1 Context and Operational Need

The Detailed User Access Review (DUAR) process generates regular extracts of user permissions across the DevOps platform. Its purpose is to provide the Security and IAM teams with reliable data to identify obsolete, excessive or unjustified accesses.

At the time of the migration, the DUAR scope covered around ten DevOps applications, including SGitHub Enterprise, Jenkins, JFrog Artifactory, TeamCity, SonarQube, ArgoCD, XLDeploy, Nexus and Elasticsearch. Since each platform had its own API, authentication mechanism and user model, the existing process was difficult to maintain through independent jobs.

### 4.2.2 Existing Architecture and Its Limits

Before the migration, the DUAR process was implemented through several independent Jenkins jobs, usually one job per application. Although this approach worked

initially, it became difficult to maintain as the number of platforms increased.

- **Duplicated logic:** authentication, error handling and output formatting were repeated across several jobs.
- **Limited extensibility:** adding a new application required creating or adapting a dedicated Jenkins job.
- **Fragmented observability:** logs and execution statuses were scattered across different Jenkins jobs, making daily monitoring more difficult.
- **Maintenance effort:** API changes or data model updates had to be applied separately for each job, increasing the risk of inconsistent behavior.

The objective of the migration was therefore not only to move from Jenkins to Airflow, but also to redesign the process around a more modular, observable and configuration-driven architecture.

### 4.2.3 Target Architecture: Dynamic DAG on Apache Airflow

The new architecture is based on a dynamic Airflow DAG approach. Instead of maintaining one independent Jenkins job per application, a common DAG structure is generated from a centralized configuration. Each application keeps its own connector, but the global workflow remains the same.

The project was organized with a clear separation of responsibilities:

```
dags/
  perform_duar.py

utils/
  connectors/
    connector_for_each_application.py

  generate_extract_file/
    create_duar_file.py
    get_cdpusers.py
    userscsv.py

  data_ingestion/
    ingest_data.py

  s3_storage/
    s3_storage.py

  alerting.py
  mailing.py
```

**Figure 4.1:** DUAR Airflow project structure

The main DAG file, `perform_duar.py`, defines the Airflow workflow and orchestrates the different steps of the process. The `connectors/` directory contains the platform-specific extraction logic. Each connector is responsible for retrieving users and permissions from one DevOps application.

The `generate_extract_file/` package contains the logic used to standardize extracted data into the expected DUAR format. It also includes the matching with CDP users, which is required to enrich platform accounts with internal identity information. The `s3_storage/` package handles the storage of generated files, while `data_ingestion/` manages the submission of the final extract to the downstream data platform. Finally, `alerting.py` and `mailing.py` are used to notify the team in case of execution failure.

This structure makes the solution easier to maintain. Adding a new application mainly requires adding a new connector and its configuration, without rewriting the whole DAG logic.

### 4.2.4 Execution Flow

Each generated DUAR workflow follows the same execution sequence:

| Step | Purpose |
|---|---|
| Extract | Retrieve users and access data from each DevOps application through its connector. |
| Enrich | Match extracted accounts with CDP identity information. |
| Generate | Create a standardized DUAR extract file. |
| Store | Upload the generated extract to S3-compatible storage. |
| Ingest | Send the extract to the downstream data platform. |
| Notify | Send alerts or emails in case of failure. |

**Table 4.2:** Execution flow of a DUAR Airflow workflow

### 4.2.5 Benefits of the Migration

The migration from Jenkins to Airflow improved the DUAR process in several ways.

- **Centralized orchestration:** Airflow provides a clear view of each execution, including task status, logs and failures.
- **Reusable workflow logic:** the same DAG structure is reused across applications, reducing code duplication.

- **Easier onboarding:** new applications can be integrated by adding a connector and configuration instead of creating a complete job from scratch.
- **Better observability:** failures are easier to identify because each step of the workflow is visible in Airflow.
- **Improved maintainability:** shared components such as file generation, storage, ingestion and notification are centralized in reusable modules.

Overall, the new architecture transformed DUAR from a set of fragmented Jenkins jobs into a structured Airflow-based workflow. This made the process more industrialized, easier to monitor and better aligned with the team's DevOps/SRE practices.

### 4.2.6 GitHub Enterprise EMU Connector

Among the new applications integrated into the DUAR pipeline during this apprenticeship, **GitHub Enterprise EMU** required the development of a dedicated connector, as no existing extraction mechanism was available for this platform.

GitHub Enterprise EMU (Enterprise Managed Users) differs from the standard GitHub Enterprise Server in that user identities are fully managed by the enterprise's identity provider via the **SCIM 2.0 protocol** (System for Cross-domain Identity Management). The connector developed in Python implements the full SCIM pagination protocol to retrieve the complete user list, including:

- Token-based authentication against the SCIM endpoint.
- Pagination handling to retrieve all users regardless of list size.
- HTTP status validation and retry logic for transient failures.
- Field mapping from SCIM attributes to the DUAR data model.

### 4.2.7 Secrets Management and Configuration

All platform credentials are retrieved at runtime from **HashiCorp Vault**, with no credential stored in the DAG code or Airflow connection configuration Platform configurations (API endpoints, ingestion parameters, data model mappings) are stored in **Airflow Variables**, providing a centralized and UI-accessible configuration layer that can be updated without code changes or DAG redeployment.

### 4.2.8 Results

## 4.3 Leaver Access Revocation Automation

| Indicator | Outcome |
|---|---|
| Platforms covered | ∼10 DevOps applications |
| New connectors developed | GitHub Enterprise EMU (SCIM 2.0) + scope extensions |
| Architecture | 1 DAG factory → 1 DAG per platform, config-driven |
| Execution frequency | Daily (automated Airflow scheduler) |
| Secrets management | HashiCorp Vault (zero hard-coded credentials) |
| Observability | Per-task Airflow logs + Alerta alerts + email notifications |
| Data destination | My BigData platform (validated ingestion ID: 740653) |

**Table 4.3:** DUAR migration — summary of outcomes

### 4.3.1 Continuity with the Master 1 Internship

The leaver access revocation pipeline was first designed and implemented during the M1 internship (2024), covering five platforms: Jenkins, TeamCity, JFrog Artifactory, SGitHub Enterprise, and Nexus. The modular Python architecture and GitHub Actions orchestration introduced at that stage are documented in the M1 report and are not reproduced here.

The M2 apprenticeship contribution is threefold: **regular production execution** of the pipeline for already-covered platforms, **extension of the automation scope** to three additional platforms, and **deepened integration** with the Security team's workflow and ITSM change governance.

### 4.3.2 Production Execution: Scale and Operational Impact

Throughout the apprenticeship, leaver revocation changes were executed regularly in production. Table 4.4 reports representative batch sizes from recent change operations, illustrating the scale at which the automation operates.
The contrast with the manual process is significant. On a multi-instance platform such as Jenkins, manual revocation requires an engineer to identify on which instance(s) each user holds an account, navigate to each instance, and delete accounts individually. For a batch of 100+ users, this represents several hours of focused engineering time. The automated pipeline completes the equivalent operation in under two minutes.

### 4.3.3 Scope Extension: SonarQube, ArgoCD and XLDeploy

During the apprenticeship, the automation scope will be extended to three additional platforms:

| Platform | Users revoked (example batch) | Automated execution time |
|---|---|---|
| JFrog Artifactory | 300 | $<$ 2 minutes |
| Jenkins (all instances) | 102 | $<$ 2 minutes |
| TeamCity | 88 | $<$ 2 minutes |
| SGitHub Enterprise | 120 + 63 (2 changes) | $<$ 2 minutes |

**Table 4.4:** Leaver revocation — representative batch sizes and automated processing times

- **XLDeploy and ArgoCD**: both platforms expose REST user management APIs. The connector modules were developed following the same interface as existing connectors (`delete_users()` function), and deployed to production.
- **SonarQube**: connector development and testing in progress at the time of writing. SonarQube presents a specific challenge: user accounts are partially managed through LDAP synchronization, requiring careful distinction between locally-managed and directory-synchronized accounts to avoid revocation conflicts.

### 4.3.4 Security Team Workflow and ITSM Integration

The operational workflow for each leaver change follows a structured sequence that integrates the Security team, the change management process, and the automation pipeline:

**Step 1.** The Security team provides a leaver list (Excel file) containing the identifiers of users to revoke, validated against the DUAR extract.

**Step 2.** A Unity change request is created, documenting the scope (platforms, estimated user count), the execution method (automated pipeline), and the rollback procedure.

**Step 3.** The change is reviewed and approved by the change management board.

**Step 4.** The GitHub Actions pipeline is triggered (manually or scheduled), processes the leaver list per platform, and generates structured execution logs.

**Step 5.** Logs are archived as GitHub Actions artifacts and attached to the Unity change record. The change is closed with post-implementation evidence.

**Step 6.** A synchronization meeting with the Security team validates outcomes and addresses any anomalies (accounts not found, partial failures).

## 4.4 Airflow-Based Operational Workflow Automation

### 4.4.1 File Storage Backup Automation

#### Context and Importance

File storage backups represent a foundational reliability requirement for any production platform. In the DevOps/SRE context, the applications hosted on the platform — Jenkins, XLDeploy, Jira, GitHub Actions Runners — store critical operational data: CI/CD job configurations, deployment packages, project management data, and runner cache. Loss of this data due to storage failure or accidental deletion without a consistent backup can result in significant service disruption and recovery time.

The backup solution implemented uses **Azure File Storage snapshots**, a point-in-time capture mechanism that preserves a consistent state of the file share at the moment of snapshot creation. The challenge is that applications writing to file storage must be in a **consistent state** at the time of the snapshot — i.e., they must not be in the middle of a write operation that would leave the backup in an inconsistent state. This requirement, known as **backup consistency**, necessitates that the backup process be orchestrated rather than simply scheduled.

#### DAG Architecture

Four Airflow DAGs were developed, one per application: **Jenkins** (all instances), **XLDeploy**, **Jira**, and **GitHub Actions Runners**. Each DAG follows a structured task sequence designed to ensure backup consistency:
Key design decisions:

- **Consistency group handling**: applications deployed across multiple regions or instances are backed up as a consistency group, ensuring that all instances are snapshotted at the same logical point in time.
- **Per-application scheduling**: each DAG has an independent schedule aligned with the application's usage pattern and recovery point objective (RPO).
- **Failure alerting**: snapshot failures trigger immediate notifications, as an unnoticed backup failure can compromise the organization's recovery capability.

### 4.4.2 Bond Weather Portal: Operational Data Pipeline

#### Context

**Bond Weather** is an internal web portal developed by Société Générale to provide a unified, real-time operational dashboard for all DevOps platform squads. It

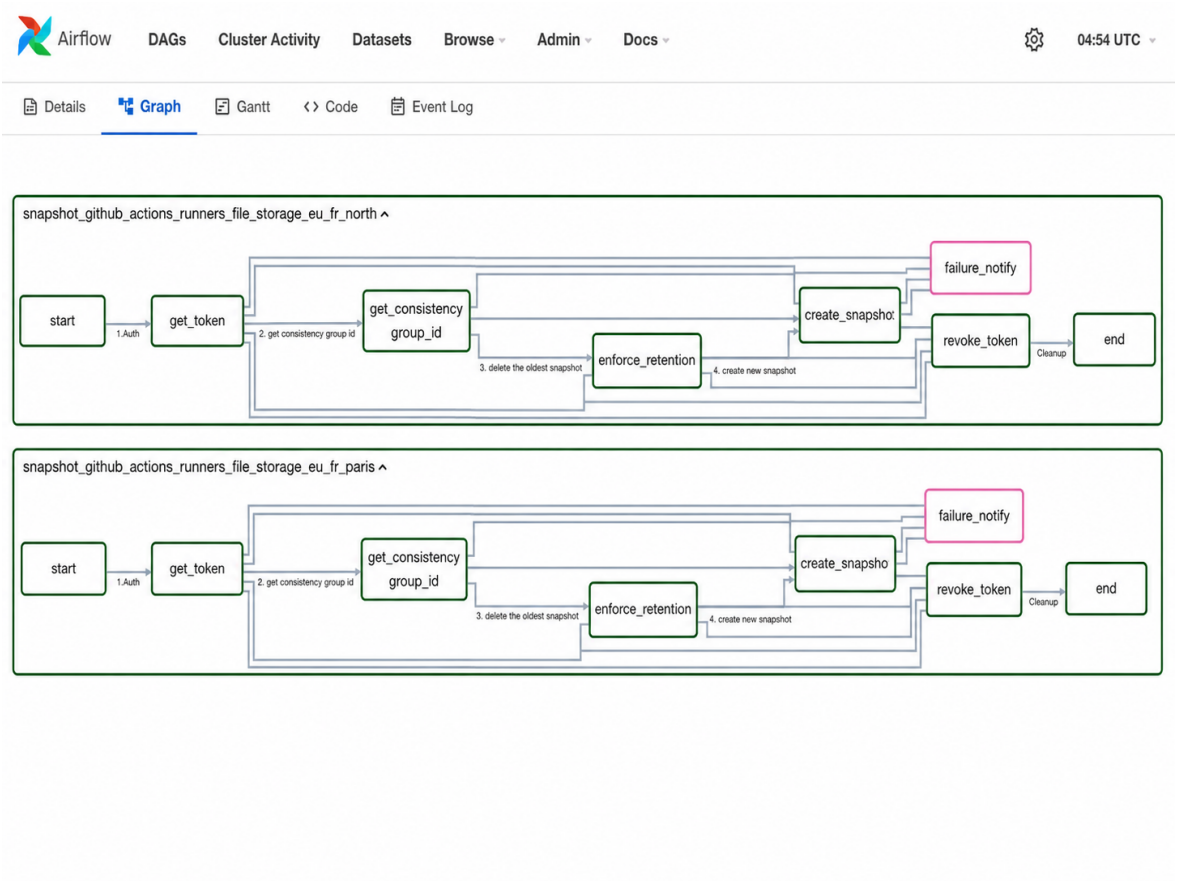


**Figure 4.2:** Anonymized Airflow graph view for file storage backup DAGs

centralizes four categories of operational information:

- **Upcoming production changes**: all planned changes across all squads, sourced from the Unity ITSM system.
- **Ongoing incidents**: active incident records affecting platform services.
- **Problems**: open problem records under investigation.
- **Recent changes**: recently closed change records for post-implementation review.

The portal is built on a **Strapi** backend (a headless CMS used as an API layer) and consumed by a frontend UI. Before the implementation of the Airflow pipeline, the data feeding the portal was updated manually or not updated at all, leading to stale and unreliable information on the dashboard.

#### Implemented Solution: Four Airflow DAGs

Four Airflow DAGs were designed and deployed to feed the Bond Weather portal automatically:

**Changes DAG..** Queries the Unity ITSM API for all production change records within a defined time window (upcoming changes within the next N days, and recently closed changes). Filters, structures, and synchronizes the data to the Strapi backend. Scheduled to run multiple times daily to ensure near-real-time accuracy of the upcoming changes view.

**Incidents DAG..** Retrieves active incident records from Unity, filters by severity and affected platform scope, and synchronizes them to Strapi. Provides the "ongoing incidents" view visible to all squads on the Bond Weather dashboard.

**Problems DAG..** Fetches open problem records from Unity and synchronizes them to Strapi. Problems represent known issues under root-cause investigation and are tracked separately from incidents.

**Cleaner DAG..** Removes change records older than 30 days from the Strapi database to prevent data accumulation and maintain portal performance. Scheduled to run daily.

All four DAGs follow the same integration pattern: Unity ITSM API query $\rightarrow$ data transformation and filtering $\rightarrow$ Strapi REST API upsert. Authentication to both Unity and Strapi uses service account credentials managed via Airflow Connections and Vault.

#### Impact

Prior to this pipeline, the Bond Weather portal displayed incomplete or outdated information, reducing its usefulness as an operational tool. After deployment, the portal became a reliable, continuously updated source of operational truth for all platform squads, reducing the need for manual status queries during incidents and change windows.

### 4.4.3 Archiving DAGs for JFrog Artifactory (Work in Progress)

As Artifactory repositories grow over time, unused or outdated artifacts accumulate, consuming storage and degrading repository performance. The archiving project aims to automate the artifact lifecycle management process through a set of Airflow DAGs implementing the following workflow:

**(1)** **Identify** artifacts meeting archival criteria (age, last download date, absence of active references).

**(2)** **Transfer** identified artifacts from Artifactory to an S3-compatible long-term storage bucket.

**(3)** **Clean** the Artifactory repository by removing transferred artifacts, maintaining traceability references.

**(4)** **Process in parallel** to handle large artifact sets efficiently.

The project is currently in the design and development phase. The DAG structure and orchestration logic are defined; ongoing work focuses on adapter logic between the JFrog Artifactory API and the S3 transfer layer, and on robustness testing for large-scale artifact sets.

## 4.5 CI/CD Pipeline for Airflow Deployment

### 4.5.1 Problem: Sequential and Manual Deployment Steps

The deployment of workflows and components to the Airflow instance relied on a sequence of separate operations. An artifact had to be created from a configuration, then its availability had to be checked, and the deployment had to be triggered afterwards through another call.

This approach was functional, but not optimal. It introduced waiting time between steps, made deployments harder to reproduce, and became inefficient when several environments or deployment configurations had to be managed.

### 4.5.2 Proposed CI/CD Approach

The objective of this contribution is to design a unified CI/CD pipeline for Airflow deployment. The pipeline is intended to structure the process into two main stages: artifact creation and artifact deployment.

The **CI stage** focuses on creating a deployable artifact from a defined configuration. This configuration describes the source repositories, Python dependencies, system dependencies and storage resources required by the Airflow workload. The result of this stage is an artifact identifier that can be reused by the deployment stage.

The **CD stage** uses this artifact identifier to deploy the generated artifact to the target Airflow environment. The deployment can include environment-specific settings such as variables, secrets management through Vault, synchronization options and target environment selection.

**Configuration** $\longrightarrow$ **Artifact creation** $\longrightarrow$ **Artifact ID** $\longrightarrow$ **Deployment to Airflow** $\longrightarrow$ **Monitoring**

**Figure 4.3:** Target CI/CD workflow for Airflow artifact deployment

### 4.5.3 Expected Benefits and Current Status

This contribution is still in progress, but its expected benefits are clear. The pipeline aims to reduce the need for manual chaining of API calls, improve deployment consistency across environments, and make Airflow deployments more reproducible.

Once completed, the solution will provide a more structured deployment process for Airflow workloads, aligned with DevOps practices such as automation, traceability and environment-aware deployment.

## 4.6 Infrastructure Modernization

### 4.6.1 Context and Technical Challenge

As part of the team's infrastructure modernization roadmap, a **Windows Server 2016** virtual machine was migrated from **Generation 1 (Gen1)** to **Generation 2 (Gen2)** on the Société Générale private cloud infrastructure. The distinction between VM generations is not cosmetic: Gen1 VMs use a legacy BIOS firmware and emulated hardware model, while Gen2 VMs use UEFI firmware, support Secure Boot, and offer improved boot performance and hardware compatibility. Critically, this is not an in-place upgrade — it requires building a new VM from scratch and migrating all data, configurations, and services.

In a regulated banking environment, this type of infrastructure change carries significant risk: any misconfiguration during the migration can cause service disruption for the teams depending on the VM, with potential impact on CI/CD pipelines or operational workflows. The full ITSM change lifecycle was applied: pre-migration documentation, development environment validation, formal change approval, production cutover during a maintenance window, and post-implementation verification.

### 4.6.2 Migration Process and Outcome

The migration was executed in four phases:

**Phase 1.** **Inventory**: full documentation of the Gen1 VM (hardware allocation, installed software, network configuration, running services, dependent pipelines).

**Phase 2.** **Gen2 provisioning and configuration**: creation and configuration of the target Gen2 VM with equivalent resource allocation, UEFI firmware, and required software stack.

**Phase 3.** **Data and service migration**: transfer and validation of all application data and service configurations.

**Phase 4.** **Production cutover**: execution during a planned maintenance window, with real-time rollback readiness.

The migration was completed successfully without service interruption. Beyond the immediate technical outcome, this mission provided practical exposure to **legacy infrastructure constraints in a banking context**: the gap between cloud-native immutable infrastructure practices and the operational reality of long-running VMs maintained represents a persistent challenge for SRE teams in large organizations.

CHAPTER 5

# Results, Critical Analysis and Perspectives

This chapter summarizes the main results obtained during the apprenticeship, their operational impact, the remaining limits of the implemented solutions, and the perspectives for future improvements.

## 5.1 Main Results and Operational Impact

The contributions presented in Chapter 4 produced measurable improvements in access governance, operational reliability and SRE toil reduction. The main results are summarized in Table 5.1.

| Area | Main result |
|---|---|
| DUAR migration | Daily access review generation for around ten DevOps applications through an Airflow-based workflow. |
| Leaver revocation | Automated revocation of user accounts across several platforms, with representative batches processed in under two minutes. |
| Operational workflows | Airflow DAGs developed for file storage backups, Bond Weather data synchronization and artifact lifecycle automation. |
| CI/CD for Airflow | Design of a structured CI/CD approach for Airflow artifact creation and deployment. |
| Infrastructure modernization | Participation in the migration of a Windows Server virtual machine from Gen1 to Gen2 in the private cloud environment. |

**Table 5.1:** Consolidated operational results across the contributions

From a security perspective, the most significant impact concerns access governance. The DUAR migration improved the visibility of user permissions across the DevOps toolchain, while the leaver revocation automation reduced the manual effort required to remove obsolete accounts. These two contributions are complementary: DUAR

helps identify access situations that must be reviewed, while the leaver pipeline supports the remediation of accounts that should no longer remain active.

From an SRE perspective, the work contributed to reducing toil. Several recurring or manual tasks were moved into automated workflows, making them more repeatable, observable and easier to control. Table 5.2 summarizes the main before/after improvements.

| Task | Before | After |
|---|---|---|
| Leaver revocation | Manual or semi-manual processing across several platforms. | Automated execution through a GitHub Actions pipeline with structured logs. |
| DUAR generation | Independent Jenkins jobs with fragmented monitoring. | Airflow-based workflow with daily scheduling and task-level observability. |
| File storage backups | Manual or partially automated backup execution. | Scheduled Airflow DAGs with snapshot creation and failure notification. |
| DAG deployment | Sequential or manual deployment steps. | Target CI/CD approach based on artifact creation and controlled deployment. |

**Table 5.2:** Before/after comparison of key automated workflows

Beyond time savings, the main value of these automations is operational control. Executions are no longer only dependent on manual actions: they produce logs, execution traces, artifacts or references that can be reviewed afterwards. This is particularly important in a banking environment, where automation must remain compatible with auditability and ITSM change management.

## 5.2 Limits and Remaining Risks

Although the implemented solutions improved several operational processes, some limits remain.

**Incomplete platform coverage..** The leaver revocation automation does not yet cover all possible platforms with the same maturity level. Some connectors are already in production, while others remain in progress or require additional validation. As long as coverage is incomplete, manual follow-up may still be required for some tools.

**Dependency on input data quality..** Both DUAR and leaver revocation depend on the quality of upstream data. If a source platform returns incomplete data, or if a leaver list contains formatting errors or missing identifiers, the automation may produce incomplete results. This means that automated workflows still require input validation and post-execution review.

**Automation risk..** Automating security-sensitive operations reduces human error, but also introduces new risks. A wrong configuration, an incorrect input file or an API behavior change could lead to an incorrect execution. For this reason, the implemented solutions must remain controlled through peer review, dry-run or test execution when possible, ITSM approval, and clear execution logs.

**Operational dependency on platforms..** The new workflows rely on platforms such as Airflow, GitHub Actions, Vault and internal APIs. If one of these components is unavailable, some automated processes may be delayed. This is a normal trade-off when moving from manual operations to centralized automation, but it must be considered in the operational model.

These limits do not invalidate the benefits of the implemented work. They show that automation in a regulated environment must be treated as an engineering system: it must be maintained, monitored, tested and continuously improved.

## 5.3 Perspectives

Several improvements can extend the work carried out during the apprenticeship.

**Complete platform coverage..** A first perspective is to finalize the remaining connectors and extend the leaver revocation scope to all relevant DevOps platforms. This would reduce the need for manual remediation and make the process more homogeneous.

**Industrialize Airflow deployment..** The CI/CD pipeline for Airflow deployment should be completed in order to provide a unified workflow from artifact creation to deployment. This would improve reproducibility across environments and reduce manual chaining of API calls.

Overall, the main perspective is to continue moving from manual operational actions toward automated, observable and auditable workflows, while keeping the governance requirements of the banking context at the center of the design.

CHAPTER 6

# Conclusion

## 6.1 Conclusion

This report presented the work carried out during the Master 2 apprenticeship within the DevOps/SRE team of Société Générale, following a previous Master 1 internship in the same environment. The main objective was to contribute to the automation of identity governance and operational security processes in a regulated banking context.

The contributions focused on several complementary areas. The DUAR process was industrialized through an Airflow-based approach, improving the generation and monitoring of access review data across DevOps applications. The leaver revocation automation reduced manual effort and improved traceability when removing obsolete user accounts. Additional Airflow DAGs were developed for operational workflows such as file storage backups and Bond Weather data synchronization. The work also included the design of a CI/CD approach for Airflow deployment and participation in infrastructure modernization activities.

These contributions answer the problem statement of the report by showing that automation can reduce human error, improve operational efficiency and strengthen traceability, provided that it remains integrated with existing governance processes. In this context, automation does not replace control. It supports control by making executions more repeatable, more observable and easier to audit.

The work also highlighted that automation is never completely finished. It depends on data quality, platform coverage, API stability, monitoring and continuous maintenance. The implemented solutions therefore represent a solid foundation, but they must continue to evolve with the DevOps platform and the security requirements of the organization.

## 6.2 Skills and Outlook

This apprenticeship allowed me to strengthen both technical and professional skills. From a technical perspective, I gained practical experience with Python automation, Apache Airflow, GitHub Actions, REST API integration, secrets management, access governance and ITSM-controlled production changes. I also improved my understanding of how DevOps/SRE practices are applied in a large banking environment, where reliability, security and traceability are essential constraints.

A particularly important learning point was the difference between developing a script that works and building an automation that can be safely used by a team in production. The second requires modularity, logs, error handling, peer review, documentation, validation and governance integration.

During the apprenticeship, I also followed an internal GitOps training program. This training reinforced my understanding of modern delivery practices, especially the role of Git as a source of truth, the importance of declarative deployment, and the use of CI/CD pipelines to make infrastructure and application delivery more controlled and reproducible. This knowledge directly complements the work carried out on Airflow deployment and operational automation.

Looking forward, the most relevant continuation of this work would be to complete the remaining automation scope, strengthen monitoring and validation, and explore the use of DUAR data for access analytics. This would allow the platform to move from descriptive access review toward more proactive detection of risky or abnormal access patterns.

Overall, this apprenticeship confirmed my interest in DevOps/SRE, automation and security engineering. It gave me the opportunity to work on real production problems and to understand how technical decisions must be aligned with operational risk, team practices and regulatory constraints.

# References


[1] National Institute of Standards and Technology, "Security and privacy controls for information systems and organizations (NIST SP 800-53, rev. 5)," NIST, Tech. Rep., 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-53r5.

[2] OWASP Foundation. "OWASP top ten 2021 — a01: broken access control," Accessed: May 1, 2025. [Online]. Available: https://owasp.org/Top10/A01_2021-Broken_Access_Control/.

[3] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. [Online]. Available: https://sre.google/sre-book/table-of-contents/.

[4] European Parliament and Council, "Regulation (EU) 2022/2554 on digital operational resilience for the financial sector (dora)," Official Journal of the European Union, Tech. Rep. L 333, 2022, Entered into force January 17, 2025. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32022R2554.

[5] Scaled Agile Inc. "Scaled agile framework (SAFe) 6.0," Accessed: May 1, 2025. [Online]. Available: https://scaledagileframework.com.

APPENDIX A

# Risk and Control Matrix

**Table A.1:** Risk and control matrix for automated security workflows

| Risk | Likelihood | Impact | Control / Mitigation |
|---|---|---|---|
| Incorrect user deletion (false positive) | Low | High | Dry-run mode; peer review of leaver list before execution |
| Incomplete leaver list from Security team | Medium | High | Pre-execution count check; execution blocked if file missing or malformed |
| API downtime on target platform | Medium | Medium | Retry logic; failed platforms logged and flagged for manual follow-up |
| Credential exposure in pipeline | Low | Critical | All secrets stored in GitHub Secrets; no hard-coded credentials |
| DAG failure without alerting | Medium | Medium | Airflow task failure alerts; log archival for audit |
| Scope gap (unlisted application) | Medium | Medium | DUAR scope reviewed quarterly with Security and IAM teams |
| Legacy VM migration failure | Low | High | Full pre-migration snapshot; rollback documented in Unity change |

APPENDIX B

# Before/After Workflow Comparison

| Dimension | Before (manual) | After (automated) |
|---|---|---|
| Leaver revocation time (100+ users) | Several hours per platform | $<$ 2 minutes per batch |
| DUAR report generation | Manual Jenkins job per application | Daily automated DAG per application |
| Error rate | Human errors (omissions, wrong instance) | Near-zero; instance resolved from DUAR data |
| Audit traceability | Manual ITSM entries only | Structured logs + GitHub artifacts + ITSM |
| Scope coverage | Partial, per-tool knowledge required | Full platform coverage via config registry |
| On-call effort (change nights) | High manual execution effort | Minimal, pipeline monitored |
| New application onboarding | New Jenkins job from scratch | Config entry in registry |

**Table B.1:** Before/after comparison of key operational workflows